

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



FIUSAC
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



DESARROLLO DE ANALIZADOR LÉXICO CON INTERFAZ GRÁFICA

PONDERACIÓN: 25 pts

 **Tiempo estimado: 20 hrs**

Índice

1. MARCO FORMATIVO	3
1.1. Valor.....	3
1.2. Competencia(s).....	3
1.3. Objetivo SMART.....	3
2. Resumen Ejecutivo	4
3. Enunciado del Proyecto	4
3.1 Descripción del problema a resolver.....	4
3.2 Alcance del proyecto	4
3.3 Entregables	5
4. Material de apoyo	7
5. Metodología.....	7
6. Recursos y herramientas a utilizar	8
7. Desarrollo de Habilidades Blandas.....	8
8. Cronograma.....	9
9. Rúbrica de Calificación.....	10
9.1 Requisitos para optar a la calificación	10
9.2 Resumen de Puntuaciones.....	12
9.5 Comentarios Generales.....	15
Detalle de la Calificación	15

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Excelencia	Los estudiantes demostrarán excelencia técnica al implementar un analizador léxico funcional y eficiente, aplicando los principios fundamentales de la teoría de autómatas y lenguajes formales con precisión y profesionalismo.

1.2. Competencia(s)

Tipo de Competencia	
Competencia General	Diseñar, implementar y evaluar soluciones computacionales aplicando los fundamentos teóricos y prácticos de las ciencias de la computación para resolver problemas reales en el ámbito del desarrollo de software.
Competencia Específica	Aplicar los conceptos de teoría de autómatas, lenguajes formales y expresiones regulares para desarrollar analizadores léxicos que procesen y clasifiquen tokens en código fuente, utilizando herramientas de programación modernas y buenas prácticas de desarrollo de software.

1.3. Objetivo SMART

SMART	Definición	Objetivo redactado
Específico (¿Qué?)	El objetivo es concreto y tangible.	Los estudiantes serán capaces de diseñar, implementar y probar un analizador léxico funcional con interfaz gráfica que permita definir tokens mediante expresiones regulares, procesar archivos de código fuente, generar tablas de tokens y detectar errores léxicos.
Medible (¿Cuánto?)	El objetivo tiene una medida objetiva de éxito.	Lograr un analizador que reconozca correctamente al menos el 95% de los tokens definidos en los casos de prueba proporcionados, genere reportes de errores precisos con información de línea y columna, y cuente con una interfaz gráfica intuitiva que supere una evaluación de usabilidad básica.
Alcanzable (¿Cómo?)	El objetivo debe ser posible con los recursos disponibles.	Utilizando lenguajes de programación orientados a objetos (Java, Python o C#), bibliotecas para interfaces gráficas (Swing, JavaFX, Tkinter, PyQt), expresiones regulares nativas del lenguaje, y las herramientas de desarrollo disponibles en el laboratorio o mediante instalación gratuita.

Realista (¿Para qué?)	El objetivo contribuye a metas más amplias.	Desarrollar habilidades prácticas en compiladores, fortalecer el conocimiento en teoría de autómatas y lenguajes formales, y prepararse para cursos avanzados de compiladores e interpretadores, además de adquirir experiencia en desarrollo de software con interfaz gráfica.
A Tiempo (¿Cuándo?)	El objetivo tiene fecha límite o mejor aún un cronograma de hitos de progreso	Completar el proyecto en un plazo de 4 semanas, con entregas parciales programadas: definición de requisitos y diseño, implementación del analizador léxico base, integración de interfaz gráfica y pruebas, y entrega final con documentación completa.

2. Resumen Ejecutivo

Este proyecto se enmarca en el curso de Lenguajes Formales y de Programación, específicamente en el tema de análisis léxico y teoría de autómatas. El problema identificado es la dificultad que enfrentan los estudiantes para comprender de manera práctica y visual el funcionamiento de un analizador léxico, primera fase fundamental en el proceso de compilación. La solución propuesta consiste en el desarrollo de un analizador léxico con interfaz gráfica que permita a los usuarios definir sus propios tokens mediante expresiones regulares, cargar archivos de código fuente, visualizar el proceso de tokenización, identificar errores léxicos con precisión, y opcionalmente observar el autómata finito determinista (AFD) correspondiente. El proyecto permite aplicar competencias en teoría de autómatas, expresiones regulares, estructuras de datos, programación orientada a objetos, desarrollo de interfaces gráficas de usuario y documentación técnica profesional.

3. Enunciado del Proyecto

3.1 Descripción del problema a resolver

En el desarrollo de compiladores e intérpretes, el análisis léxico constituye la primera fase fundamental del proceso de compilación. Esta etapa es responsable de transformar una secuencia de caracteres en una secuencia de tokens significativos para las fases posteriores del compilador. Sin embargo, para los estudiantes de ciencias de la computación, comprender y visualizar este proceso puede resultar abstracto sin una herramienta práctica que permita experimentar con diferentes definiciones de tokens y observar el comportamiento del analizador.

El problema que este proyecto busca resolver es la necesidad de contar con una herramienta interactiva que facilite el aprendizaje y la comprensión del análisis léxico, permitiendo a los usuarios definir sus propios tokens, procesar código fuente y visualizar tanto los resultados del análisis como la representación formal del autómata finito determinista (AFD) que reconoce dichos tokens.

3.2 Alcance del proyecto

Alcance obligatorio:

- **Gestión de tokens:** La aplicación debe permitir la definición, creación, modificación y eliminación de tokens mediante expresiones regulares o patrones específicos. Como mínimo, debe soportar la definición de 5 tipos de tokens diferentes (identificadores, números, operadores, palabras reservadas y símbolos especiales).
- **Carga de archivos:** El sistema debe permitir cargar archivos de texto plano (.txt) que contengan código fuente para ser analizado. Debe validar que el archivo exista y sea legible.
- **Análisis léxico:** El analizador debe recorrer el archivo de entrada carácter por carácter, identificando y clasificando cada lexema según los tokens definidos. Debe reconocer espacios en blanco, saltos de línea y tabulaciones como delimitadores.
- **Tabla de tokens:** Generar y mostrar una tabla que presente cada token identificado con su lexema correspondiente, el tipo de token, y la posición en el código fuente (línea y columna).
- **Detección de errores:** Identificar y reportar errores léxicos, indicando la posición exacta (línea y columna) donde se encontró el error y una descripción del problema (caracteres no reconocidos o secuencias inválidas).
- **Interfaz gráfica:** Desarrollar una interfaz gráfica de usuario (GUI) intuitiva que incluya:
 - Área de texto para visualizar el código fuente cargado
 - Panel para definir y gestionar tokens
 - Tabla de resultados del análisis
 - Área de visualización de errores léxicos

Alcance opcional:

- **Visualización del AFD:** Representar gráficamente el Autómata Finito Determinista correspondiente a los tokens definidos, mostrando estados, transiciones y estados de aceptación.
- **Exportación de resultados:** Permitir exportar la tabla de tokens y el reporte de errores en formatos como CSV, JSON o PDF.
- **Análisis en tiempo real:** Implementar análisis léxico mientras el usuario escribe en un editor de texto integrado, con resaltado de sintaxis básico.

- **Modo de comparación:** Permitir cargar múltiples conjuntos de definiciones de tokens y comparar los resultados del análisis sobre el mismo archivo fuente.
- **Estadísticas de análisis:** Mostrar estadísticas sobre la frecuencia de cada tipo de token, número total de lexemas procesados y tiempo de análisis.

4.3 Requerimientos técnicos

Los estudiantes deberán utilizar un lenguaje de programación orientado a objetos como Java, Python o C#. Para el desarrollo de la interfaz gráfica, se recomienda el uso de bibliotecas como:

- **Java:** Swing o JavaFX
- **Python:** Tkinter, PyQt o Kivy
- **C#:** Windows Forms o WPF

El proyecto debe implementar las estructuras de datos apropiadas (tablas hash, listas, árboles) para almacenar tokens y gestionar el análisis léxico de manera eficiente. Se espera el uso de expresiones regulares para la definición y reconocimiento de patrones de tokens. Adicionalmente, el código debe estar organizado siguiendo principios de diseño orientado a objetos (encapsulamiento, modularidad, separación de responsabilidades) y debe incluir control de versiones mediante Git.

3.3 Entregables

Tipo	Descripción
Código Fuente	Código completo del analizador léxico con interfaz gráfica, organizado en módulos o clases, debidamente comentado y siguiendo estándares de codificación. El código debe incluir la implementación del analizador léxico, la gestión de tokens, el manejo de archivos y la interfaz gráfica.
Manual Técnico	Documento en PDF que incluya: arquitectura del sistema, diagramas de clases, descripción de los algoritmos implementados, estructura de datos utilizadas, explicación del proceso de análisis léxico y documentación de las expresiones regulares utilizadas.

Manual de Usuario	Guía paso a paso con capturas de pantalla que explique cómo: instalar y ejecutar la aplicación, definir y gestionar tokens, cargar archivos de código fuente, interpretar la tabla de tokens generada, comprender los mensajes de error, y visualizar el AFD (si se implementó).
Repositorio GitHub	Repositorio privado que contenga todo el código fuente, commits organizados por funcionalidades, archivo README con instrucciones de instalación y uso, y documentación del proyecto.

4. Material de apoyo

- Compilers: Principles, Techniques, and Tools" (Dragon Book) - Aho, Sethi, Ullman

5. Metodología

El proyecto seguirá una metodología de desarrollo incremental con las siguientes fases:

Fase 1: Investigación y Diseño:

- Investigar los fundamentos teóricos del análisis léxico, autómatas finitos y expresiones regulares
- Estudiar la documentación de la biblioteca de interfaz gráfica seleccionada
- Analizar ejemplos de analizadores léxicos existentes para identificar mejores prácticas
- Definir la arquitectura del sistema utilizando diagramas UML (clases, secuencia, componentes)
- Diseñar el modelo de datos para almacenar tokens, lexemas y errores

Fase 2: Implementación del Motor de Análisis Léxico:

- Implementar la clase o módulo principal del analizador léxico
- Desarrollar el sistema de gestión de tokens (crear, modificar, eliminar, almacenar)
- Implementar el algoritmo de reconocimiento de tokens usando expresiones regulares
- Implementar el sistema de detección y reporte de errores léxicos con información de posición
- Desarrollar la estructura de datos para almacenar los tokens identificados
- Crear el módulo de lectura y procesamiento de archivos de entrada

6. Recursos y herramientas a utilizar

Entorno de desarrollo integrado (IDE):

- IntelliJ IDEA o Eclipse (para Java)
- PyCharm o Visual Studio Code (para Python)
- Visual Studio (para C#)

Control de versiones:

- Git y GitHub

Herramientas de diseño:

- Draw.io o Lucidchart para diagramas UML y AFD

7. Desarrollo de Habilidades Blandas

Autogestión del tiempo:

Los estudiantes deberán organizar su tiempo de manera efectiva para cumplir con las diferentes fases del proyecto. Se recomienda crear un cronograma personal detallado con hitos semanales y dedicar entre 6-8 horas semanales al desarrollo del proyecto.

Responsabilidad y compromiso:

Cada estudiante es responsable de la calidad y completitud de su trabajo. Esto incluye no solo la funcionalidad del código, sino también la claridad de la documentación y el cumplimiento de los estándares establecidos.

Resolución de problemas:

Durante el desarrollo, los estudiantes enfrentarán desafíos técnicos que requerirán investigación independiente, análisis crítico y creatividad para encontrar soluciones óptimas. Se fomenta la consulta de documentación oficial, foros técnicos y recursos académicos.

Pensamiento crítico:

Los estudiantes deberán evaluar diferentes enfoques para implementar el analizador léxico, comparar alternativas de diseño y justificar sus decisiones técnicas basándose en criterios de eficiencia, mantenibilidad y usabilidad.

8. Cronograma

El cronograma describe las etapas clave del proyecto, los plazos estimados para cada una, y el proceso de asignación, elaboración y calificación de las tareas. Los estudiantes deberán seguir este plan para asegurar que el proyecto avance de manera organizada y cumpla con los plazos establecidos. Cada fase incluye la asignación de tareas, el tiempo estimado para su elaboración, y el momento de su calificación.

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto		
Elaboración		
Calificación		

9. Rúbrica de Calificación

Tema	Descripción	Cumple (Sí/No)
Cumplimiento de la tecnología establecida	El desarrollo debe haberse realizado en Java, Python o C# con interfaz gráfica funcional (Swing, JavaFX, Tkinter, PyQt, Windows Forms o WPF).	
Uso de herramientas requeridas	El proyecto debe incluir control de versiones con Git, repositorio en GitHub con commits organizados y README completo.	
Gestión y entregas de práctica	Entrega puntual de los 3 entregables parciales: documento de diseño, analizador sin GUI, y aplicación integrada.	
Documentación obligatoria	Incluir Manual Técnico (con diagramas UML), Manual de Usuario (con capturas de pantalla), y documento de casos de prueba, todos en formato PDF.	
Pruebas y funcionalidad mínima	El sistema debe permitir: definir al menos 5 tipos de tokens, cargar archivos .txt, generar tabla de tokens con posición (línea y columna), y reportar errores léxicos con ubicación precisa.	

Rubrica

	Criterio/Nivel			Nivel de evaluación		
No.	Criterio de evaluación	Punteo maximo		Satisfactorio 100% - 61%	Necesita mejorar 60% - 0%	Punteo Obtenido
1	Habilidades	40				
1.1	Documentación Técnica (Manual Técnico)	10	Descripción	El manual técnico está completo y profesional. Incluye arquitectura del sistema claramente explicada, diagramas UML (clases, secuencia) precisos y detallados, descripción exhaustiva de algoritmos de análisis léxico, estructuras de datos utilizadas (tablas hash, listas), explicación detallada de expresiones regulares implementadas, y justificación sólida de decisiones de diseño. El documento es visualmente atractivo, bien organizado y fácil de seguir.	El manual técnico está incompleto, carece de elementos importantes como diagramas UML, la arquitectura no está clara, los algoritmos no están explicados adecuadamente, falta documentación de expresiones regulares, la presentación es desorganizada o difícil de seguir, o la calidad técnica es superficial.	
			Rango del punteo	10-6/	5-0/	
1.2	Documentación de Usuario (Manual de Usuario)	8	Descripción	El manual de usuario es completo, claro y profesional. Incluye instrucciones detalladas de instalación, guías paso a	El manual de usuario está incompleto, las instrucciones son confusas o difíciles de	

				paso con capturas de pantalla de alta calidad, explicación clara de cómo definir tokens, cargar archivos, interpretar tabla de resultados y entender errores. Incluye sección de troubleshooting y tiene formato atractivo y fácil de seguir.	seguir, faltan capturas de pantalla importantes, no explica adecuadamente las funcionalidades principales, o la calidad general no permite a un usuario utilizar la aplicación efectivamente.	
			Rango del punteo	8-5/	4-0/	
1.3	Repositorio GitHub	7	Descripción	El repositorio está excepcionalmente bien organizado con estructura de carpetas clara y lógica, commits frecuentes y descriptivos que reflejan el progreso del proyecto, README completo con descripción del proyecto, tecnologías usadas, instrucciones de instalación, uso y ejecución, archivo .gitignore apropiado, y documentación integrada. Demuestra excelente uso de control de versiones.	El repositorio está desorganizado, tiene pocos commits o commits poco descriptivos (ej. "cambios", "fix", "update"), README incompleto o ausente, falta .gitignore, estructura de carpetas confusa, o demuestra mal uso del control de versiones.	
			Rango del punteo	7-4/	3-0/	
1.4	Presentación	8	Descripción	La presentación es profesional, clara y bien estructurada cubriendo introducción, demostración, decisiones técnicas y conclusiones.	La presentación está incompleta, desorganizada o mal diseñada.	

			Rango del punteo	8-5/	4-0/	
1.5	Casos de Prueba y Documentación de Testing	7	Descripción	Incluye más de 5 casos de prueba exhaustivos y bien documentados que cubren escenarios con código correcto, múltiples tipos de errores léxicos, casos de borde (archivo vacío, caracteres especiales), y escenarios complejos. Cada caso incluye entrada clara, resultado esperado y resultado obtenido. Demuestra testing riguroso y profesional.	Los casos de prueba son insuficientes (menos de 5), están mal documentados, no cubren escenarios importantes, falta información sobre resultados esperados u obtenidos, no demuestran testing adecuado del sistema, o son superficiales y no prueban la funcionalidad real.	
			Rango del punteo	7-4/	3-0/	
	Sub-Total de Puntos	40				
2	Conocimientos	60				
2.1	Gestión de Tokens (Crear, modificar, eliminar, almacenar)	12	Descripción	El sistema permite crear, modificar, eliminar y almacenar tokens de manera intuitiva y robusta. Soporta al menos 5 tipos diferentes de tokens con expresiones regulares complejas (identificadores, números, operadores, palabras reservadas, símbolos). Incluye validación de expresiones, prevención de	La gestión de tokens es limitada o deficiente, no soporta 5 tipos diferentes de tokens, las operaciones básicas (crear, modificar, eliminar) no funcionan correctamente, hay errores frecuentes en el manejo de expresiones regulares, la	

				duplicados, persistencia de configuración, y la interfaz para gestión de tokens es clara y funcional.	validación es inexistente, o la interfaz es confusa e inusable..	
			Rango del punteo	12-7/	6-0/	
2.2	Análisis Léxico - Reconocimiento correcto de tokens	15	Descripción	El analizador léxico reconoce correctamente el 95-100% de los tokens en los casos de prueba. Maneja correctamente delimitadores (espacios, tabulaciones, saltos de línea), identifica todos los tipos de tokens definidos con precisión, procesa archivos de cualquier tamaño sin errores, el algoritmo es eficiente, y demuestra implementación técnica sólida basada en expresiones regulares.	El analizador reconoce menos del 80% de los tokens correctamente, tiene errores frecuentes en la identificación de lexemas, no maneja bien los delimitadores, falla con archivos de prueba básicos, el rendimiento es inaceptablemente lento, o la implementación no utiliza adecuadamente expresiones regulares.	
			Rango del punteo	15-9/	8-0/	
2.3	Tabla de Tokens - Generación con lexema, tipo y posición	10	Descripción	La tabla de tokens se genera correctamente con todas las columnas requeridas: lexema identificado, tipo de token asignado, número de línea y número de columna en el código fuente. La información es 100% precisa en todos los casos de prueba. La tabla es visualmente clara, bien formateada, fácil	La tabla está incompleta (falta lexema, tipo de token o información de posición), la información de línea y columna es incorrecta frecuentemente, no se mantiene la precisión en la ubicación de los tokens, la presentación es confusa o difícil	

				de interpretar y puede incluir funcionalidades adicionales como ordenamiento o búsqueda.	de leer, o no se genera correctamente para todos los archivos de entrada.	
			Rango del punteo	10-6/	5-0/	
2.4	Detección y Reporte de Errores Léxicos	10	Descripción	El sistema detecta todos los errores léxicos correctamente (caracteres no reconocidos, secuencias inválidas), reporta la posición exacta (línea y columna) sin fallos, proporciona mensajes de error claros y descriptivos que explican el problema específico, maneja múltiples errores en un mismo archivo, y la presentación de errores es clara y profesional.	El sistema no detecta errores léxicos importantes, las posiciones reportadas son frecuentemente incorrectas o ausentes, los mensajes de error son confusos, genéricos o inexistentes, no maneja adecuadamente archivos con múltiples errores, o falla al procesar código con errores básicos.	
			Rango del punteo	10-6/	5-0/	
2.5	Interfaz Gráfica - Funcionalidad y Usabilidad	10	Descripción	La interfaz gráfica es intuitiva, profesional y completamente funcional. Incluye todos los componentes requeridos: área de visualización del código fuente, panel para definir y gestionar tokens, tabla de resultados del análisis, y área de visualización de errores léxicos. El diseño es coherente, los controles están bien etiquetados, hay retroalimentación visual	La interfaz está incompleta (faltan componentes importantes como panel de tokens o área de errores), tiene problemas de funcionalidad significativos, los botones o controles no funcionan correctamente, el diseño es confuso o poco profesional, la	

				apropiada y excelente experiencia de usuario. La navegación es fluida y lógica.	usabilidad es deficiente haciendo difícil utilizar la aplicación, o la interfaz no responde adecuadamente.	
			Rango del punteo	10-6/	5-0/	
2.6	Funcionalidades Opcionales (Visualización AFD, exportación, etc.)	3	Descripción	Implementa al menos una funcionalidad opcional de manera correcta y funcional: visualización gráfica del AFD mostrando estados, transiciones y estados de aceptación; exportación de resultados a CSV, JSON o PDF; análisis en tiempo real mientras el usuario escribe; o estadísticas de análisis con frecuencia de tokens y tiempo de procesamiento.	No implementa funcionalidades opcionales, o las que implementa están incompletas, no funcionan correctamente, o no agregan valor real al proyecto.	
			Rango del punteo	3-2/	1-0/	
	Sub-Total de Puntos	60				
	Total	100				